



A Hands-on Introduction to Multimodality

Antonio Rueda-Toicen

Senior Developer Relations Manager - EMEA

Agenda

- Intro to Multimodality
- Image Embeddings
- Contrastive Language-Image Pretraining (CLIP)
- UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction
- Generative Multimodality with Nemotron 3 Omni

Notebooks

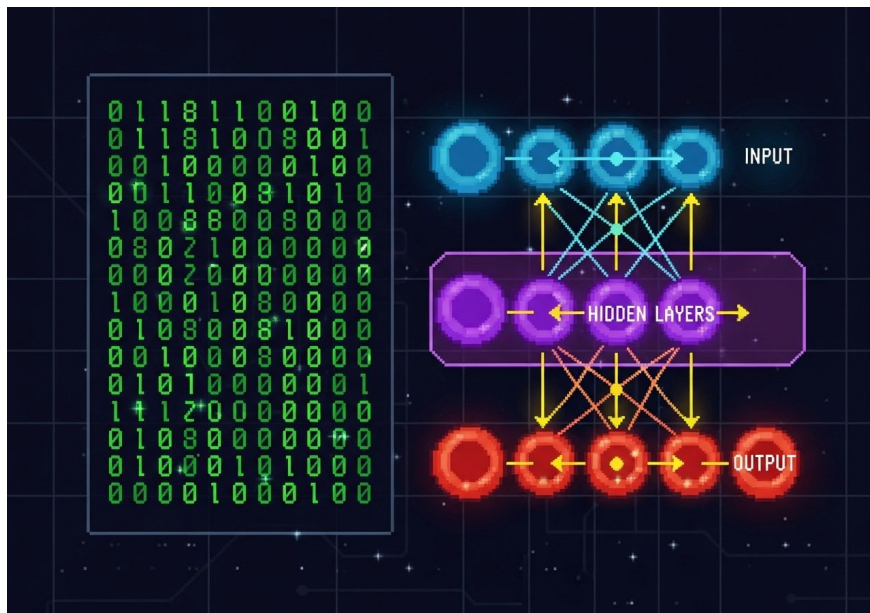
- Creating Embeddings from ResNet34
- Intro to CLIP and Zero Shot Classification
- Understanding UMAP with HyperView
- Using Nemotron 3 Omni + Nemotron Parse 1.2 for Multimodal OCR

Setup

- Configuring your NVIDIA API KEY to use models from build.nvidia.com

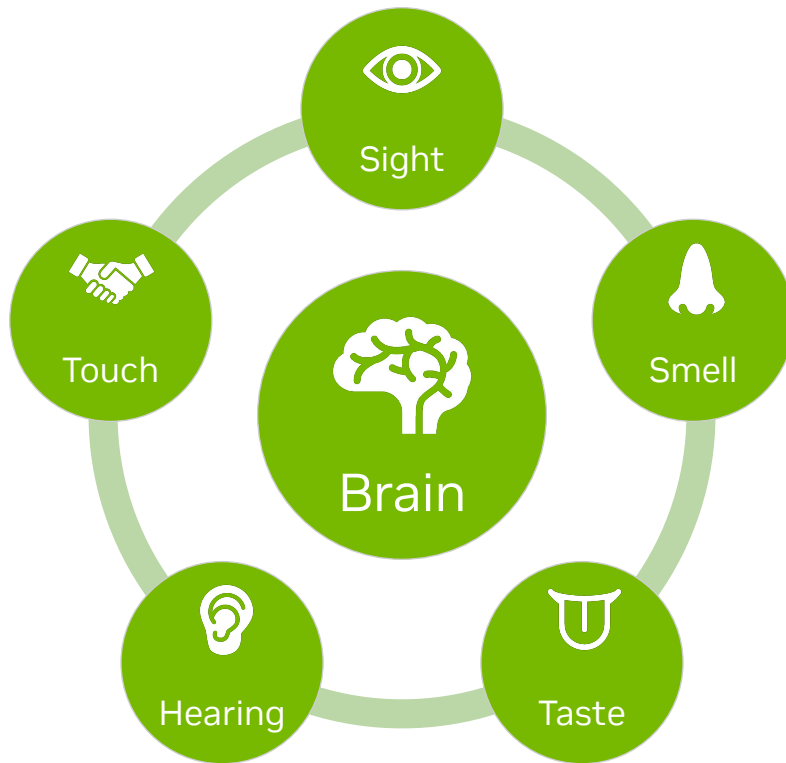
Working Hypothesis

Any data source that can be turned into a matrix can be used to train and run inference on a neural network



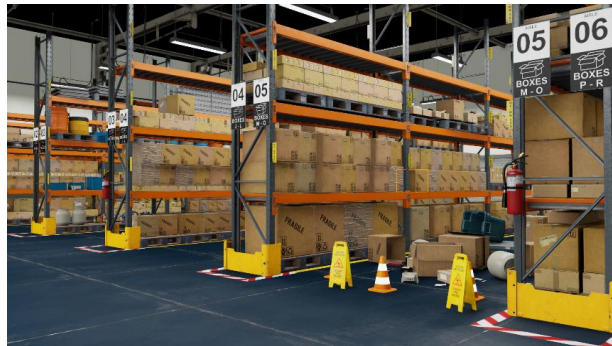
The Five Senses

How Humans Perceive

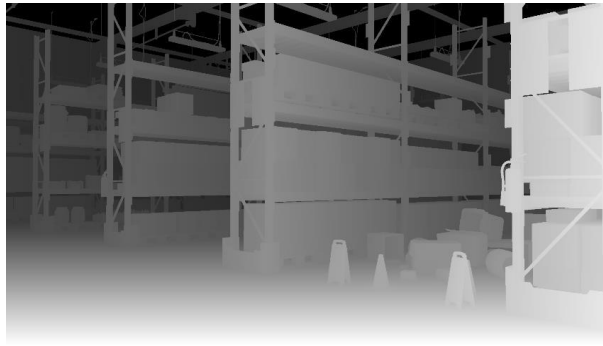


Sight

How A Computer Might See



RGB



Depth



Fisheye

Smell and Taste

How A Computer Might Smell and Taste

pH



Nutrition Facts

Nutrition Facts	
1 serving per container	
Serving size	140 g
Amount per serving	
Calories	100
% Daily Value *	
Total Fat 0.5g	1%
Saturated Fat 0g	0%
Trans Fat 0g	
Cholesterol 0mg	0%
Sodium 0mg	0%
Total Carbohydrate 24g	9%
Dietary Fiber 6g	22%
Total Sugars 15g	
Includes 0g Added Sugars	0%
Protein <1g	
Vitamin D 0mcg	0%
Calcium 0mg	0%
Iron 0.3mg	2%
Potassium 120mg	2%
* The % Daily Value (DV) tells you how much a nutrient in a serving of food contributes to a daily diet. 2,000 calories a day is used for general nutrition advice.	

Molecular Weight



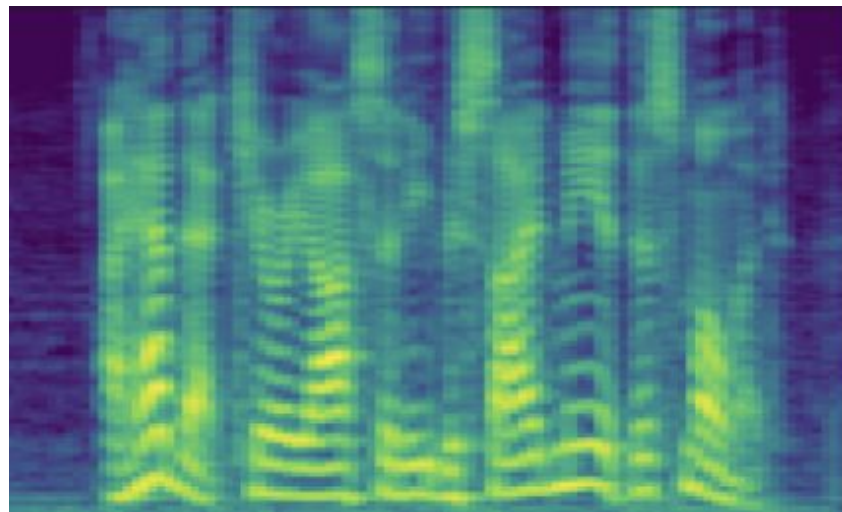
Hearing

How A Computer Might Hear

Sheet Music



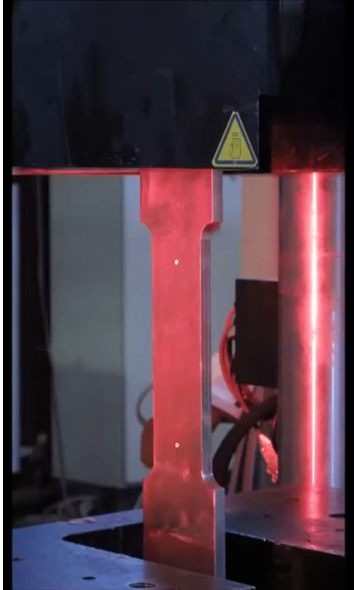
Spectrogram



Touch

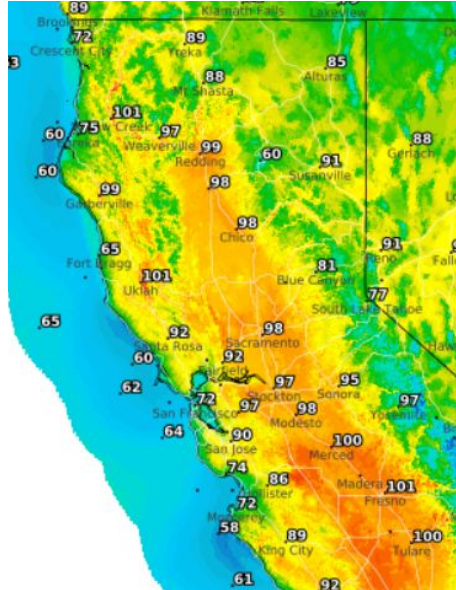
How A Computer Might Feel

Tensile Strength

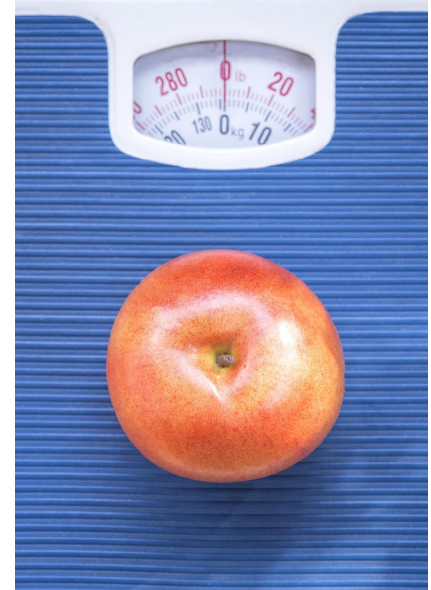


By [Christian V.](#)

Weather Maps



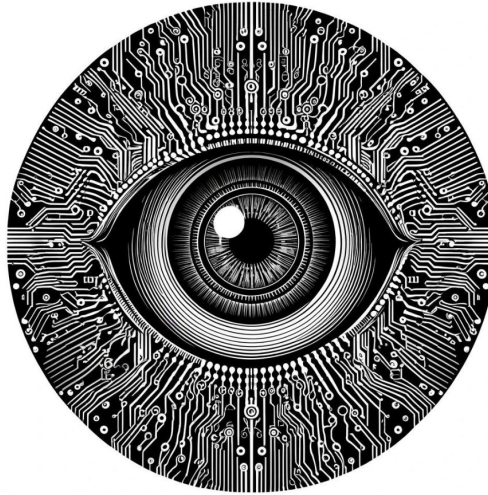
Weight





Would you call this “multi-modal”?

Image Embeddings



Learning goals

- Understand dense vector embeddings as learned representations
- Describe how image embeddings are produced in neural networks
- Extract image embeddings out of a pretrained Resnet50
- Inspect image neighborhoods using the cosine similarity metric

Sparse vs dense vector representations

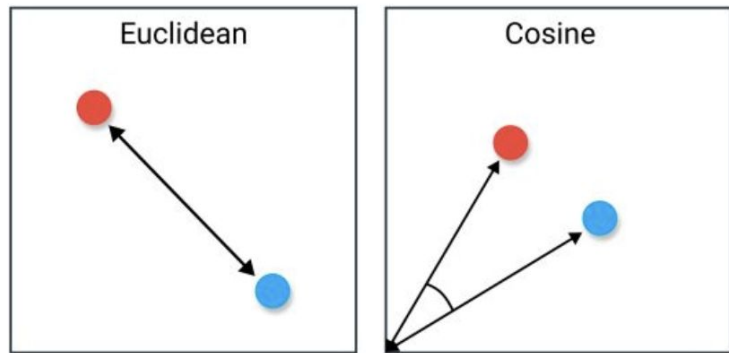
$$\mathbf{x}_1 = \text{[Image of handwritten 3]} \quad \mathbf{x}_2 = \text{[Image of handwritten 3]} \quad 28 \times 28 \text{ pixels} = 784 \text{ values}$$

$$\mathbf{x}_1 \text{ sparse} = \mathbf{x}_2 \text{ sparse} = [0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^\top$$

$$\mathbf{x}_2 \text{ dense} = [0.10 \quad -0.75 \quad 0.35 \quad 0.41 \quad -0.20 \quad 0.89 \quad -0.60 \quad 0.03 \quad 0.59 \quad -0.50]^\top$$

$$\mathbf{x}_1 \text{ dense} = [0.12 \quad -0.85 \quad 0.37 \quad 0.44 \quad -0.22 \quad 0.90 \quad -0.63 \quad 0.01 \quad 0.58 \quad -0.49]^\top$$

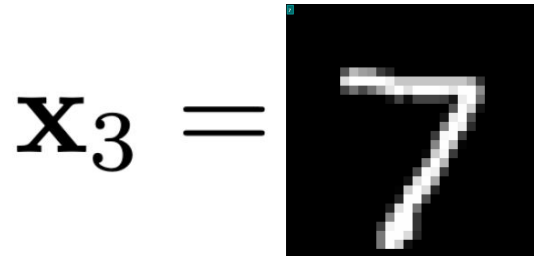
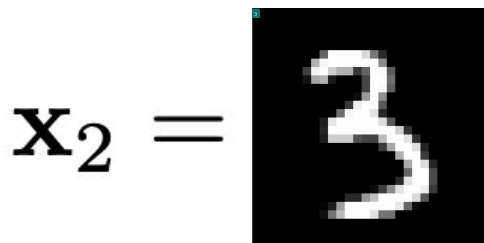
Common metrics to compare embeddings



$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

Similar objects have similar embeddings



Similar digits (X_1, X_2): High cos similarity (≈ 0.98), Low Euclidean distance (≈ 0.3)

Different digits (X_1, X_3): Low cos similarity (≈ 0.4), High Euclidean distance (≈ 2.1)

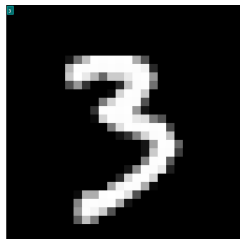
Embeddings as representations learned during training

```
import torch.nn as nn
```

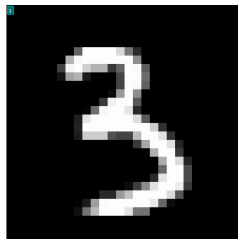
```
conv_linear_embedder = nn.Sequential(  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 64, kernel_size=3), # 24x24x64  
    nn.ReLU(),  
    nn.Flatten(), # 24*24*64  
    # This layer can be extracted as 64-dimensional  
    # embedding  
    nn.Linear(24*24*64, 64),  
    nn.ReLU(),  
    # This layer can be a 10-dimensional embedding  
    nn.Linear(in_features=64, out_features=10)  
)
```

$$H(p, q) = - \sum_i p(i) \log q(i)$$

$\mathbf{x}_1 =$



$\mathbf{x}_2 =$



$\mathbf{x}_3 =$

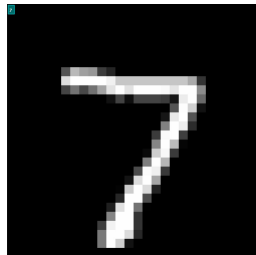
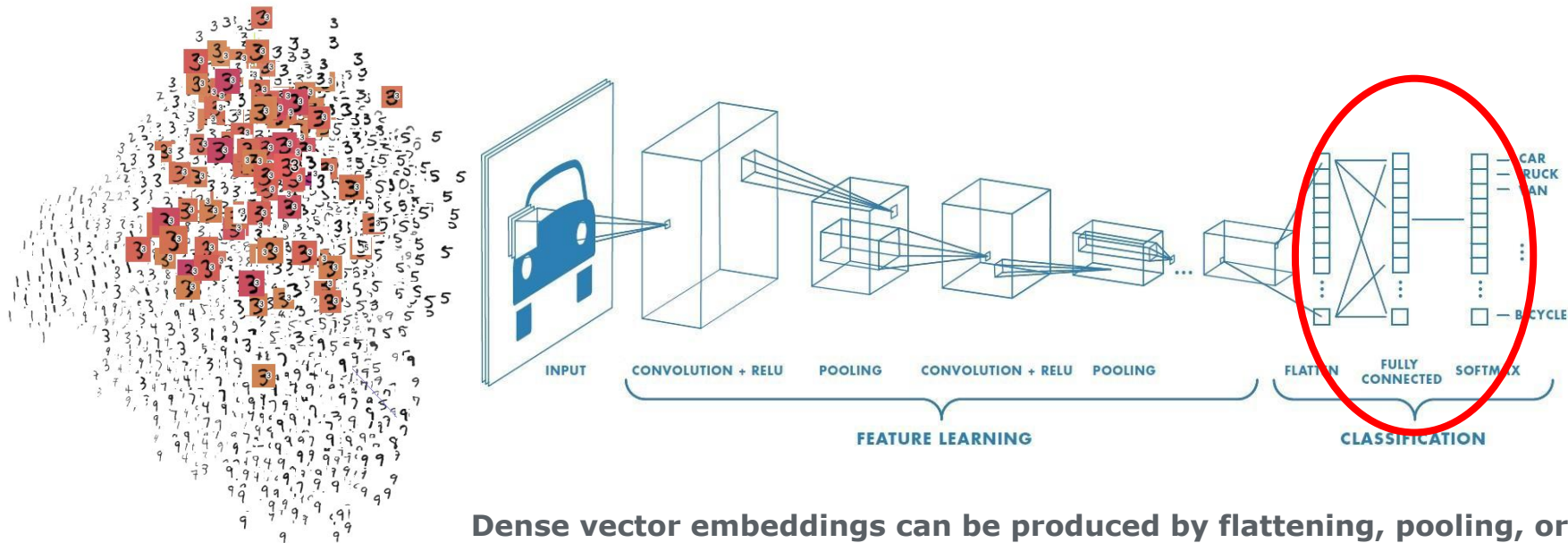


Image embeddings as a byproduct of training

When we train a neural network, we make it **embed** inputs that are semantically similar, close to each other



Dense vector embeddings can be produced by flattening, pooling, or feeding (learned) features to linear layers

[Source of image](#) (try exploring interactively)

Approaches to produce embeddings with neural networks

```
import torch.nn as nn
```

```
# Approach 1: Direct embedding
```

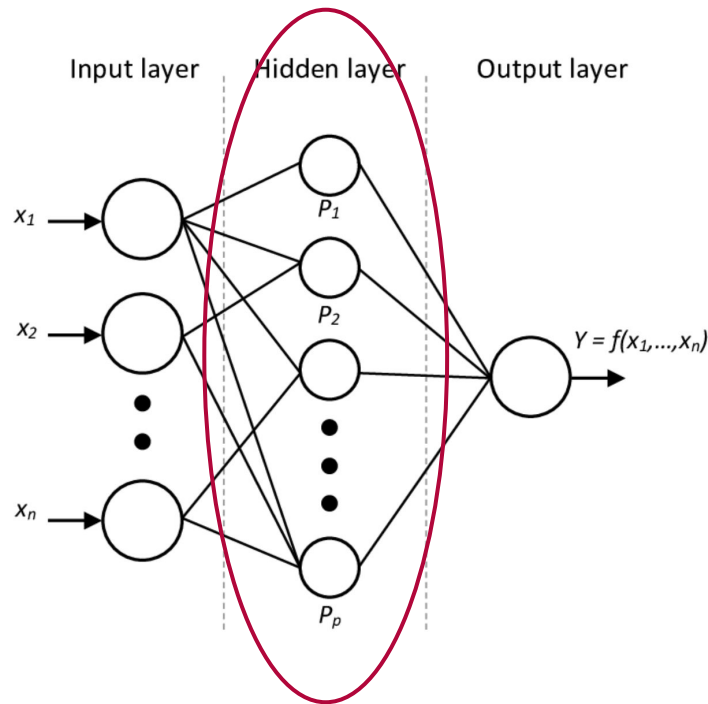
```
linear_embedder = nn.Sequential(  
    nn.Flatten(), # 784  
    nn.Linear(784, 10)  
)
```

```
# Approach 2: Convolution and then fully connected layer
```

```
conv_linear_embedder = nn.Sequential(  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 64, kernel_size=3), # 24x24x64  
    nn.ReLU(),  
    nn.Flatten(), # 24*24*64  
    nn.Linear(24*24*64, 10)  
)
```

```
# Approach 3: Global pooling after convolutions
```

```
conv_pool_embedder = nn.Sequential( # MNIST input is size 28x28  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 10, kernel_size=3), # 24x24x10  
    nn.ReLU(),  
    nn.AdaptiveAvgPool2d((1, 1)), # 1x1x10  
    nn.Flatten() # 10  
)
```



$$L_{L1} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Extract embeddings from a pretrained Resnet50

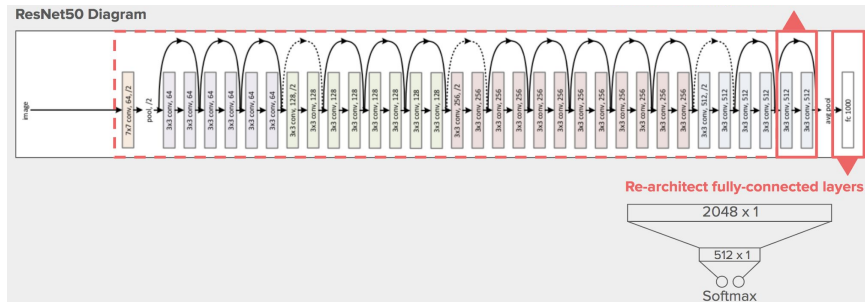
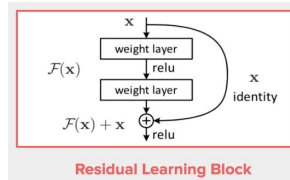
```
import torch
from torchvision import models
```

```
# Load pre-trained ResNet50 weights and transforms
weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)
# Preprocess the image using the transformations from the imagenet dataset
preprocess = weights.transforms()
```

```
# Remove final Imagenet classification layer, keep 2048 dense vector
# Take time to unpack this syntax in the practical notebook
embedder = torch.nn.Sequential(*list(model.children())[:-1])
```

```
# Disable BatchNormalization
embedder.eval()
# Disable gradient computation
with torch.inference_mode():
```

```
    # unsqueeze(0) adds the batch size for inference
    # squeeze() removes the singleton dimensions from the output vector
    embedding = embedder(preprocess(image).unsqueeze(0)).squeeze()
```



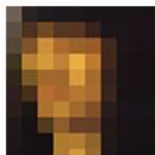
README.md

Art Recommendation system

This is the repository of a portfolio project at [DSR](#). This project aims to identify similar images using pre-trained computer vision networks. For an explanation of the technology see the [technology section](#).

Contributors

- [Catarina Ferreira](#)
- [Gargi Maheshwari](#)

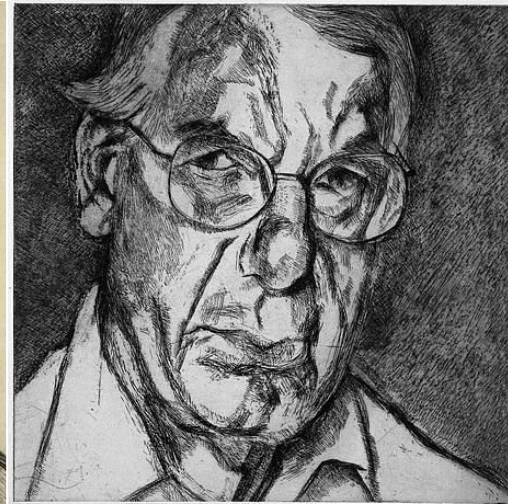


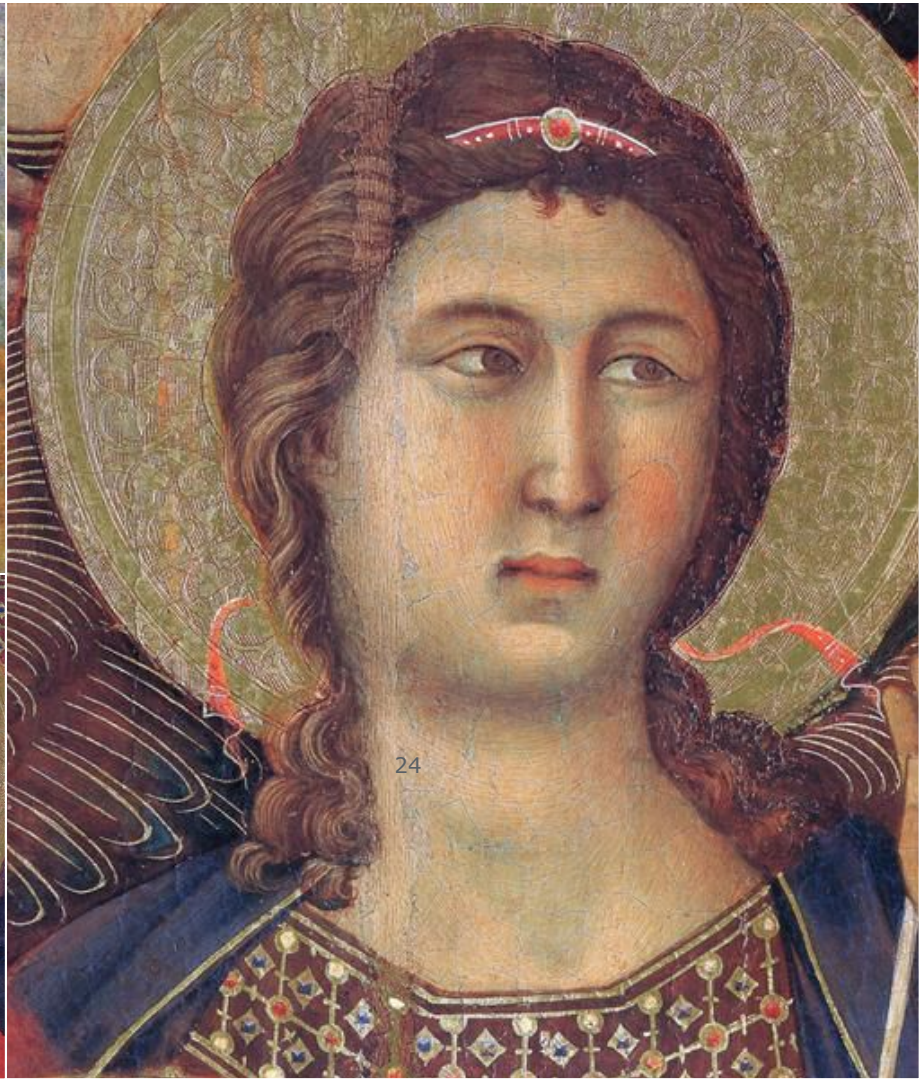
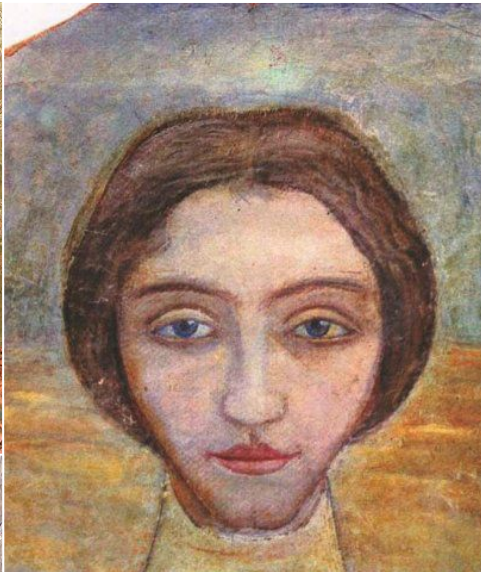
WIKIART
VISUAL ART ENCYCLOPEDIA

<https://github.com/gargimaheshwari/Wikiart-similar-art>





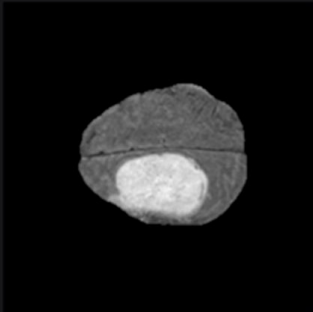




Application: differential diagnosis

PIXEL DIAGNOSE

Select Image



Cancer Type:

- ☒ Gliomas
- ☒ Meningiomas
- ☒ Metastasis

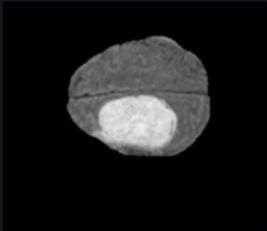
Image Type:

- ☐ t1
- ☐ t1c
- ☐ t2
- ☒ flair
- ☐ seg_t1
- ☐ seg_t1c
- ☐ seg_t2
- ☐ seg_flair

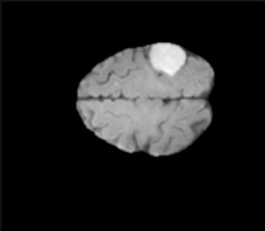
Number of Results per Type:

9

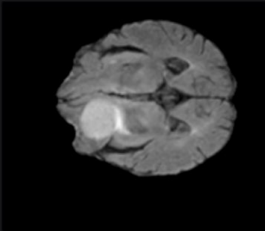
Start Search



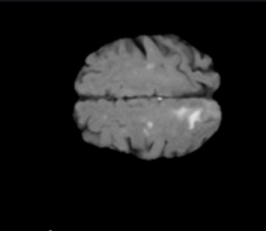
Slice no.: 112 / 150
Similarity Score: 0.9999999



Slice no.: 114 / 150
Similarity Score: 0.8529618



Slice no.: 62 / 150
Similarity Score: 0.82390577



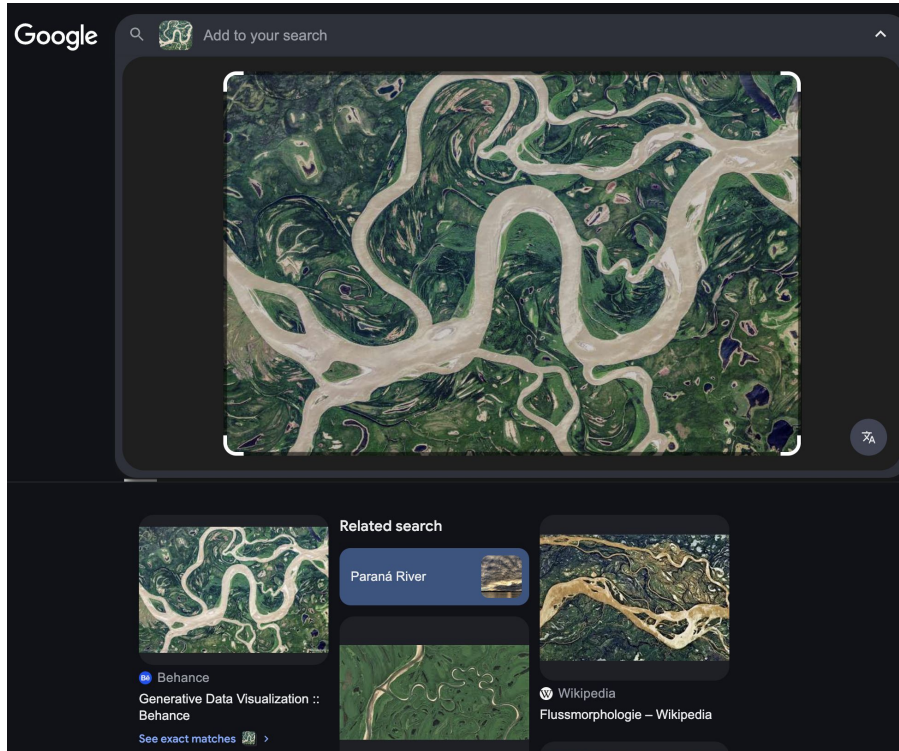
Slice no.: 109 / 150
Similarity Score: 0.8092099



Slice no.: 119 / 150
Similarity Score: 0.8026103

Image from <https://pixel-diagnose.github.io/>

Application: search by image



Search for similar images in images.google.com

Summary

Embeddings are learned vector representations

- Neural networks learn to map similar images to similar vectors through training
- We measure the similarity of embeddings using metrics like cosine similarity and Euclidean distance

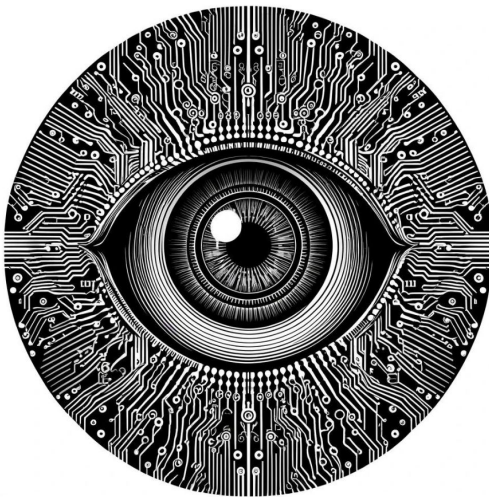
There are different architectural approaches to create embeddings

- We can use linear layers, or flattened and pooled convolutions to produce embeddings
- Imagenet-pretrained models produce rich representations in their embeddings

Embeddings have multiple applications in semantic search and dataset curation

- They can be used for reverse image search, recommendation systems, and deduplication

Contrastive Language-Image Pretraining (CLIP)



Learning goals

- Understand how CLIP models are trained
- Classify images with CLIP using a zero-shot approach
- Create image and text embeddings with a pretrained CLIP model
- Discuss CLIP's uses and limitations

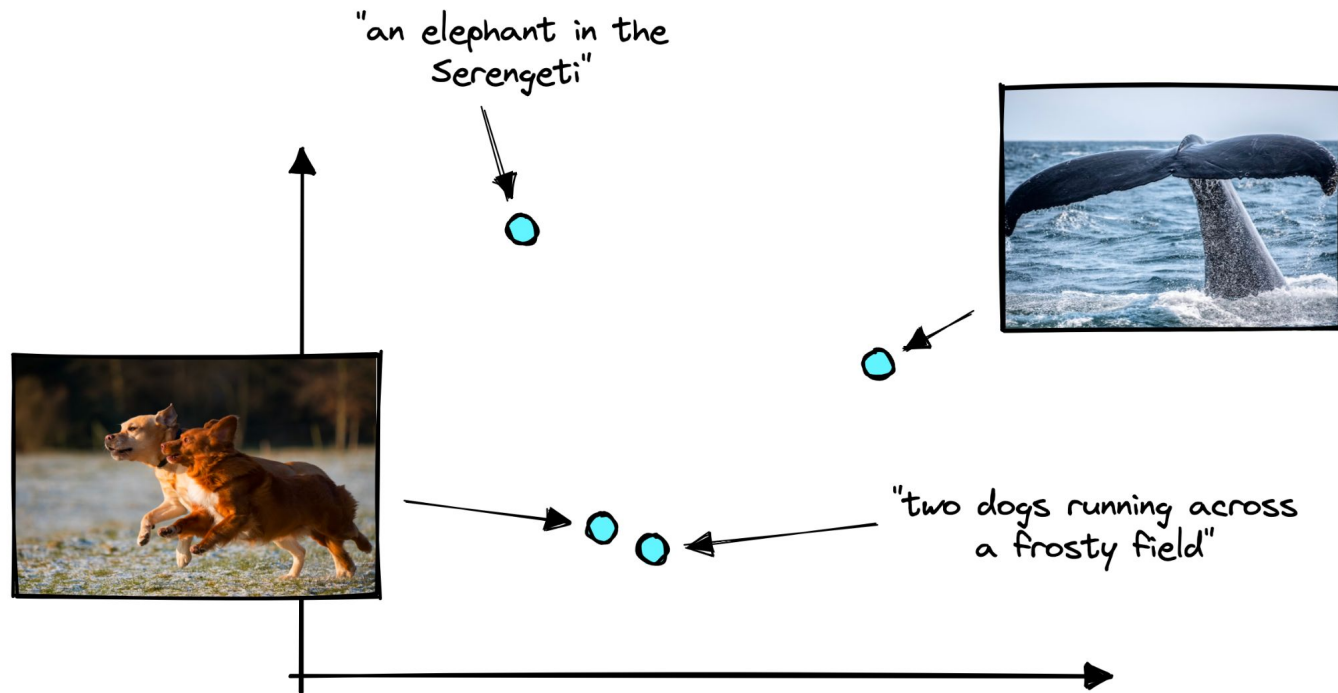
CLIP: ‘Contrastive Language Image Pretraining’

Learning Transferable Visual Models From Natural Language Supervision

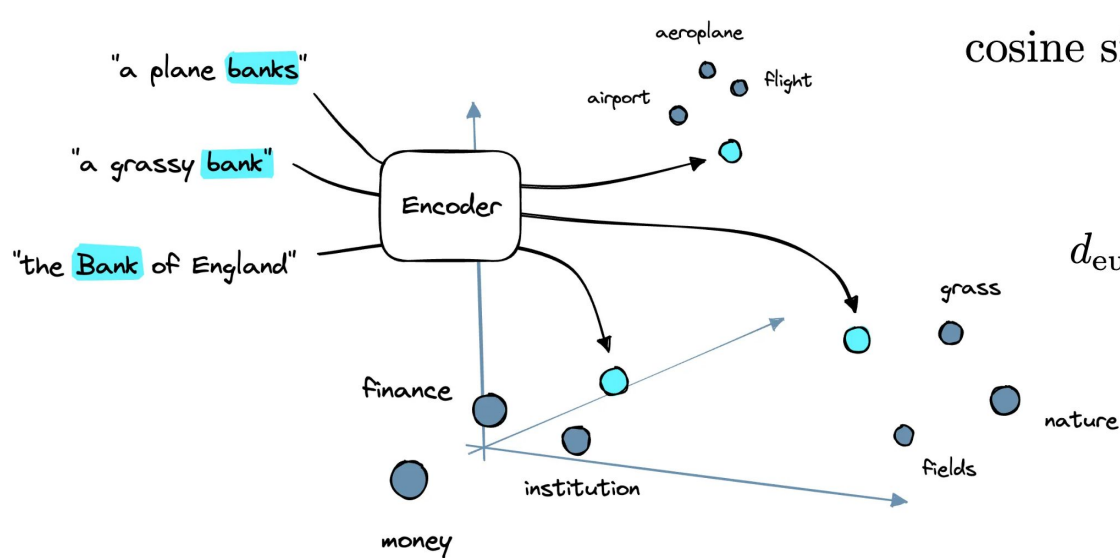
Alec Radford^{*1} Jong Wook Kim^{*1} Chris Hallacy¹ Aditya Ramesh¹ Gabriel Goh¹ Sandhini Agarwal¹
Girish Sastry¹ Amanda Askell¹ Pamela Mishkin¹ Jack Clark¹ Gretchen Krueger¹ Ilya Sutskever¹

- Connects text and images in a shared embedding space
- Created by OpenAI in 2021
- Trained on **400M image-textual description pairs** scraped from the internet
- Predicts the most relevant text snippet given an image, enabling “zero-shot classification”

Aligning text and image embeddings



Text encoders

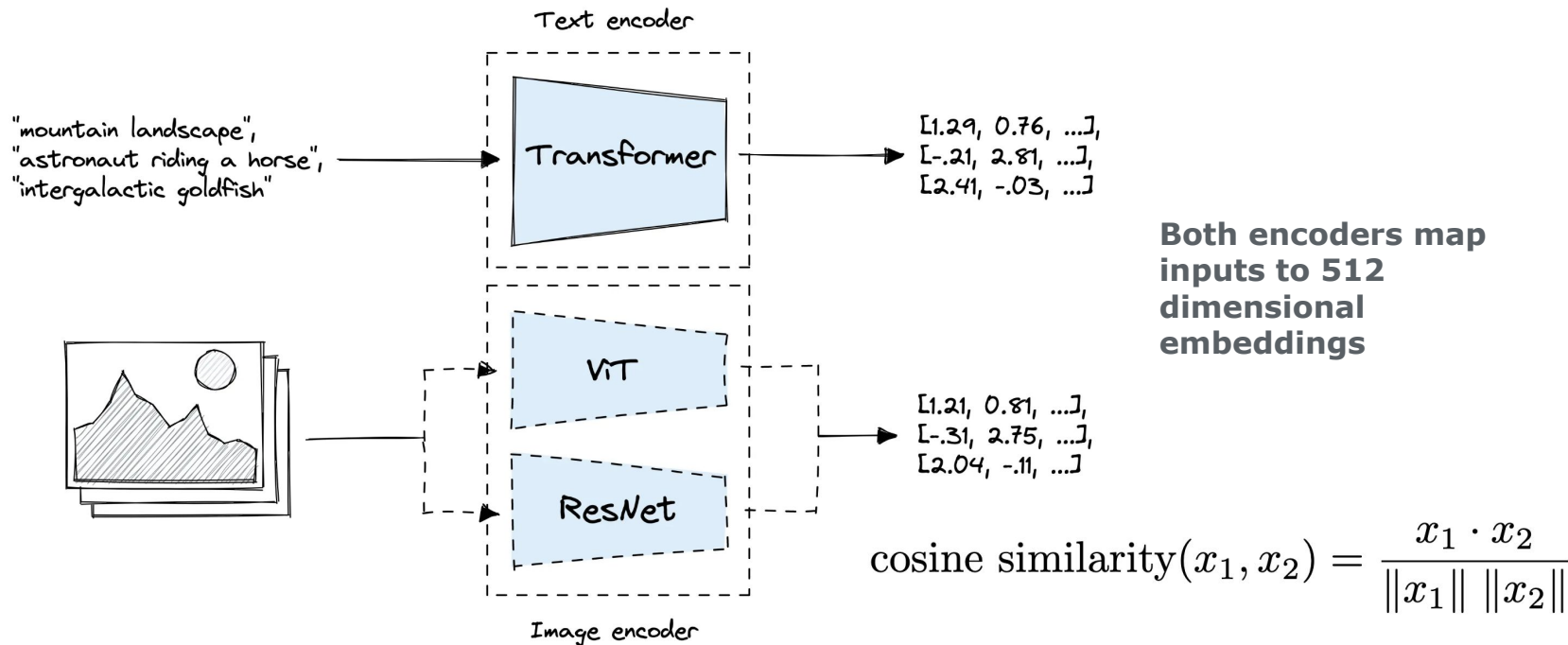


$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

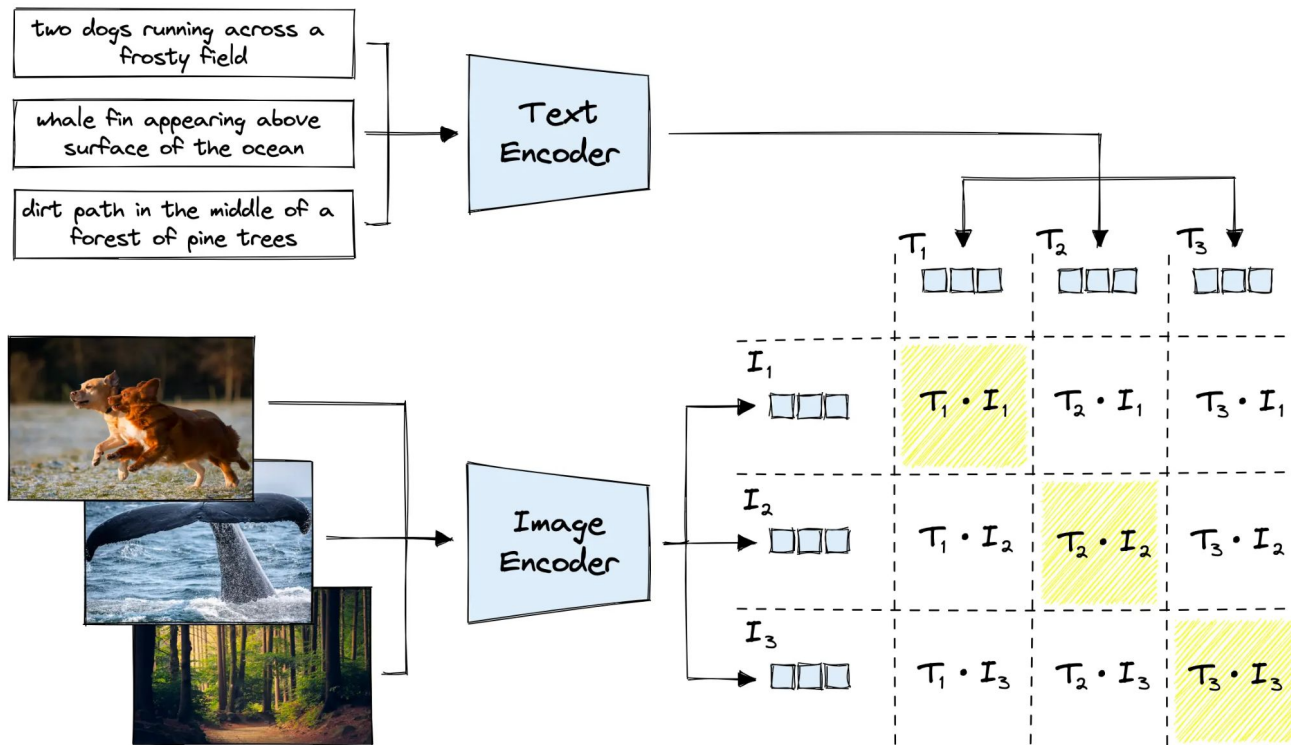
$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

Image from <https://www.pinecone.io/learn/series/image-search/vision-transformers/>

CLIP's architecture



Maximizing cosine similarity of matching text and image embeddings



Training algorithm

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

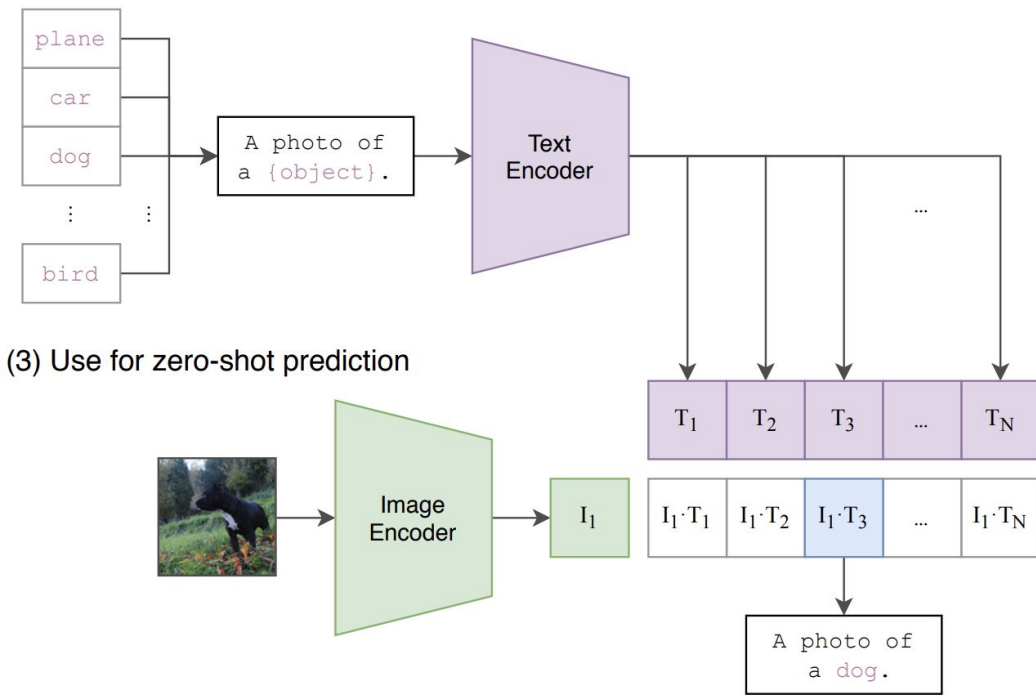
	T ₁	T ₂	T ₃	...	T _N
I ₁	I ₁ ·T ₁	I ₁ ·T ₂	I ₁ ·T ₃	...	I ₁ ·T _N
I ₂	I ₂ ·T ₁	I ₂ ·T ₂	I ₂ ·T ₃	...	I ₂ ·T _N
I ₃	I ₃ ·T ₁	I ₃ ·T ₂	I ₃ ·T ₃	...	I ₃ ·T _N
⋮	⋮	⋮	⋮	⋮	⋮
I _N	I _N ·T ₁	I _N ·T ₂	I _N ·T ₃	...	I _N ·T _N

$$H(p, q) = - \sum_i p(i) \log q(i)$$

$$H(q, p) = - \sum_i q(i) \log p(i)$$

$$\text{symmetric CE loss} = \frac{H(p, q) + H(q, p)}{2}$$

Zero-shot classification with CLIP



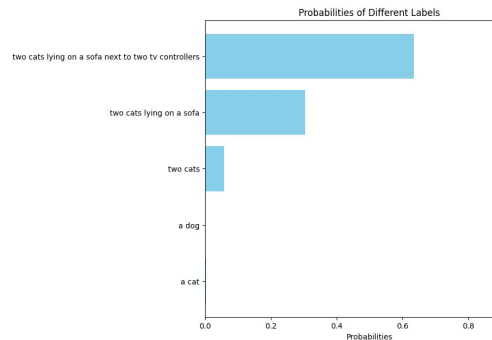
The model is able to create good classifiers without extra training

$$\text{softmax}(x_i, T) = \frac{e^{x_i/T}}{\sum_{j=1}^n e^{x_j/T}}$$

Producing embeddings with CLIP (1/2)

```
import torch
from transformers import CLIPProcessor, CLIPModel
from PIL import Image

# Load model and inputs
model =
CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
processor =
CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
image = Image.open("cats.jpg")
cat_text = ['a cat', 'a dog', 'two cats',
            'two cats lying on a sofa',
            """two cats lying on a sofa
            next to two tv controllers"""]
```



Producing embeddings with CLIP (2/2)

Get embeddings

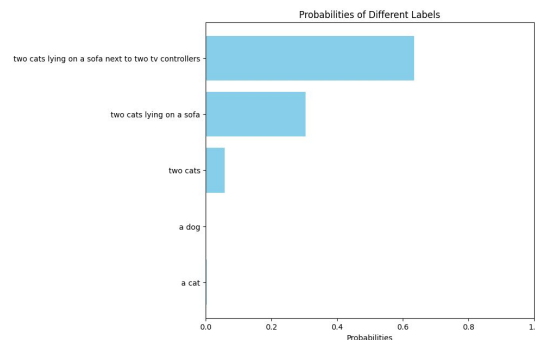
```
inputs = processor(text=cat_text, images=image, return_tensors="pt",  
padding=True)  
outputs = model(**inputs)
```

Calculate similarities

```
sims = torch.nn.functional.cosine_similarity(  
    outputs.image_embeds[:, None],  
    outputs.text_embeds[None, :],  
    dim=-1  
)
```

Print results

```
for text, sim in zip(cat_text, sims[0]):  
    print(f"{text}: {sim:.3f}")
```



Zero-shot classification with CLIP

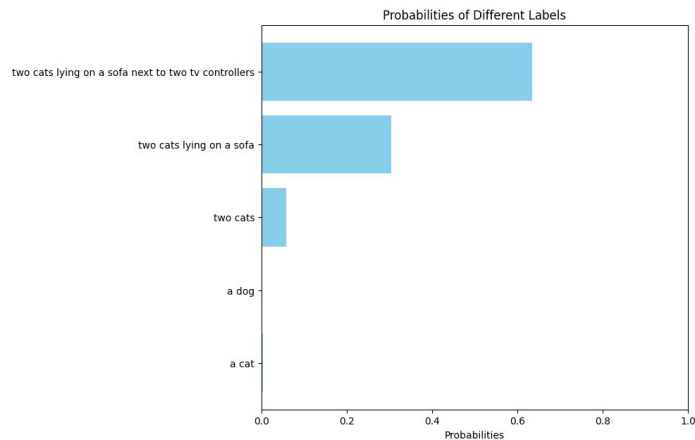
Notice what happens with the output probabilities

when we make the labels more specific

```
cat_text = ['a cat',  
            'a dog',  
            'two cats',  
            'two cats lying on a sofa',  
            'two cats lying on a sofa next to two tv controllers']
```

```
inputs = processor(text=cat_text,  
                  images=[cats_img],  
                  return_tensors="pt", padding=True)  
cat_outputs = model(**inputs)  
cat_logits_per_image = cat_outputs.logits_per_image  
cat_probs = (cat_logits_per_image/temperature).softmax(dim=1)
```

$$\text{softmax}(x_i, T) = \frac{e^{x_i/T}}{\sum_{j=1}^n e^{x_j/T}}$$



Transferable representations: CLIP against a ResNet101 pretrained on Imagenet

	Dataset Examples	ImageNet ResNet101	Zero-Shot CLIP	Δ Score
ImageNet		76.2	76.2	0%
ImageNetV2		64.3	70.1	+5.8%
ImageNet-R		37.7	88.9	+51.2%
ObjectNet		32.6	72.3	+39.7%
ImageNet Sketch		25.2	60.2	+35.0%
ImageNet-A		2.7	77.1	+74.4%

Training Resnet101: about 90 V100 GPU hours (4 days)
 Training CLIP ViT-L/14: about ~16,000 V100 GPU hours (666 days)

Note that CLIP was trained on a dataset of **400 million images** scraped from the Internet

The Imagenet1K dataset has only **1.28 million training images**

CLIP is a much more capable "foundation model"

Resnet101

~44.5 million parameters

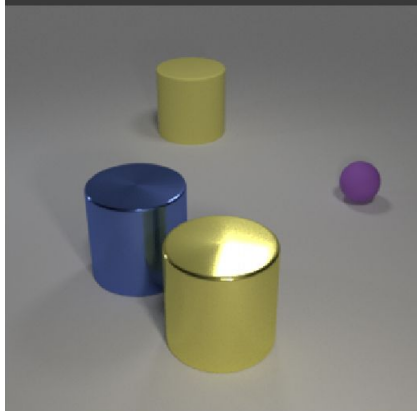
CLIP ViT-L/14@336px:

~428 million parameters

Limitations against fully supervised models

CLEVR Count

4 (75.0%) Ranked 2 out of 8 labels



× a photo of 3 objects.

✓ a photo of 4 objects.

× a photo of 5 objects.

× a photo of 6 objects.

× a photo of 10 objects.

Image from

<https://openai.com/index/clip/>

The authors comment on the limitations section of the paper that CLIP often fails to perform as well as a fully supervised model fine-tuned for the task

Notable examples:

- digit recognition (MNIST)
- tumor classification (PatchCamelyon)
- satellite imaging (EuroSAT)
- object counting (CLEVR Count)

CLIP guides image generation of diffusion models

Prompt:

"Create an image
of an astronaut
riding a horse in
pencil drawing
style"

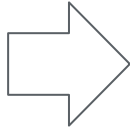


Image by OpenAI's Dall-E 3

Summary

CLIP learns joint embeddings

- CLIP creates a shared embedding space for images and text
- These multimodal embeddings have applications on semantic search and image generation

CLIP excels at zero-shot classification

- CLIP performs well on unseen tasks without additional training

CLIP has limitations

- CLIP is not always better than models fine-tuned for specific tasks
- Retraining CLIP with a ViT base is expensive both computationally and data-wise

Further reading

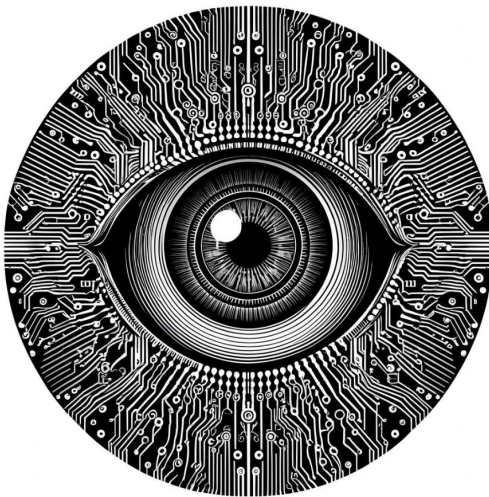
Learning Transferable Visual Models From Natural Language Supervision

- <https://arxiv.org/abs/2103.00020>

Multi-modal ML with OpenAI's CLIP

- <https://www.pinecone.io/learn/series/image-search/clip/>

Uniform Manifold Approximation and Projection (UMAP)



Learning goals

- Understand the need to project embeddings to low dimensional spaces
- Gain intuitions on UMAP



A collection of small blue and purple circles representing points in a high-dimensional space. Dashed lines connect each point to its nearest neighbors, forming a sparse network.

Step 1: Find Neighbors

Find each point's nearest neighbors in high-dimensional space.



A network of blue circles (nodes) connected by solid and dashed lines. Some connections are highlighted in green, representing a 'fuzzy' network of connections between the points.

Step 2: Build Network

Build a 'fuzzy' network representing connections between them.

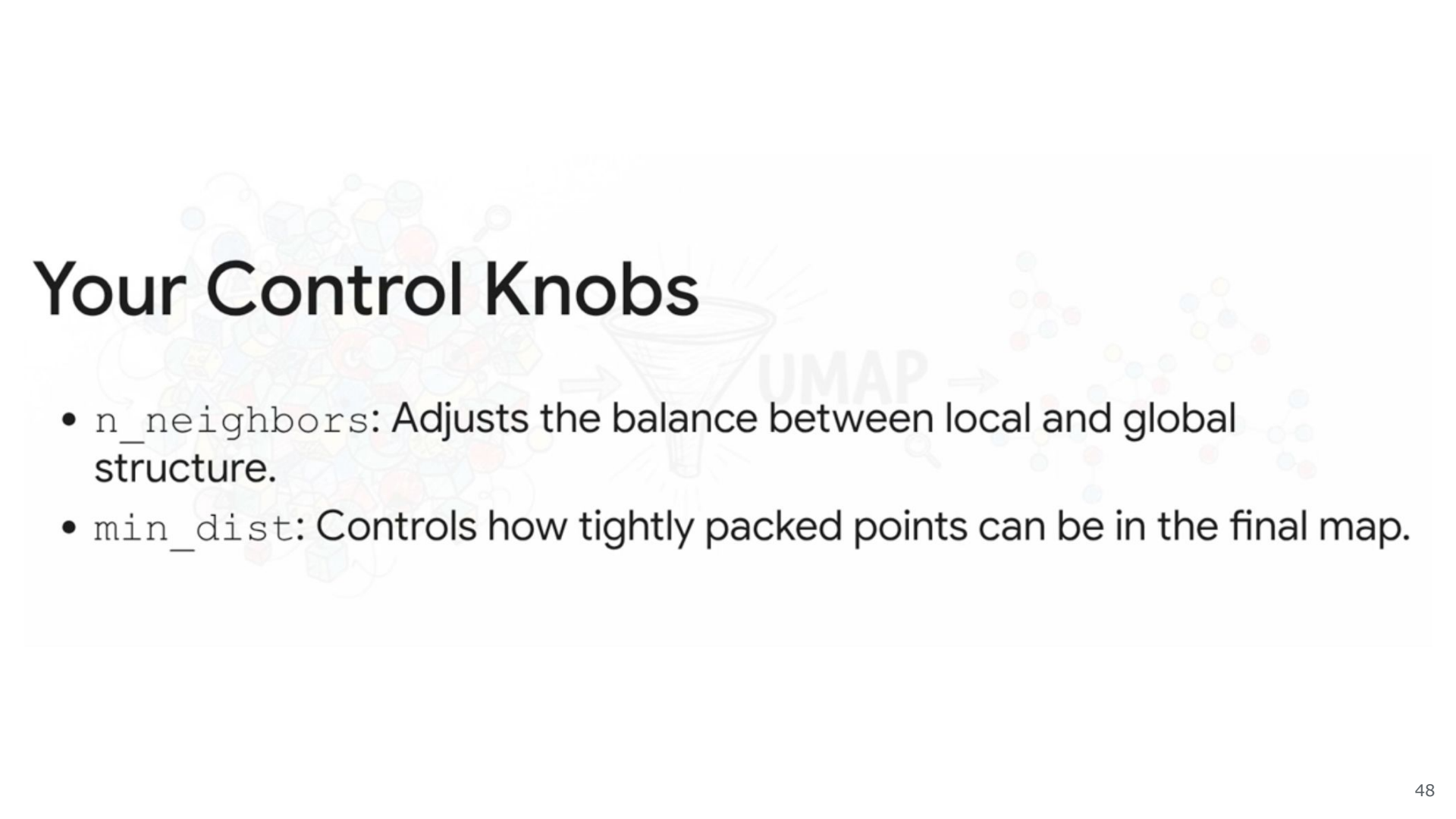


A 2D arrangement of blue circles of varying sizes, representing the final map. The circles are clustered together, with some overlapping, showing the result of a dimensionality reduction or mapping process.

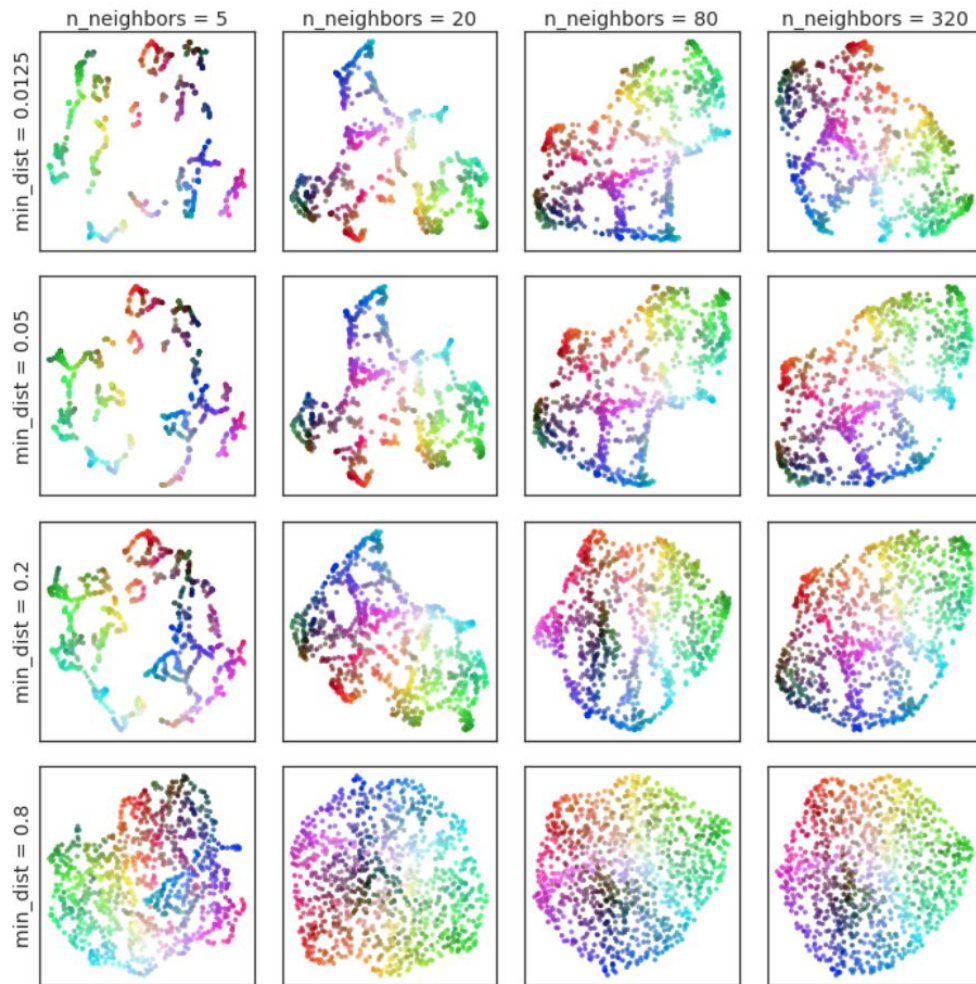
Step 3: Create Map

Find the best possible 2D arrangement of that network.

Your Control Knobs

The background features a faint, stylized illustration. In the center, there is a funnel shape with the word 'UMAP' written across it. To the left of the funnel, there is a cluster of colorful, semi-transparent geometric shapes (spheres, cubes, etc.) in shades of blue, green, yellow, and orange. To the right of the funnel, there is a network of small, colored circles connected by thin lines, resembling a molecular structure or a data graph.

- `n_neighbors`: Adjusts the balance between local and global structure.
- `min_dist`: Controls how tightly packed points can be in the final map.



UMAP: Uniform Manifold
Approximation and Projection for
Dimension Reduction

Page 24

8b73f7b370b8

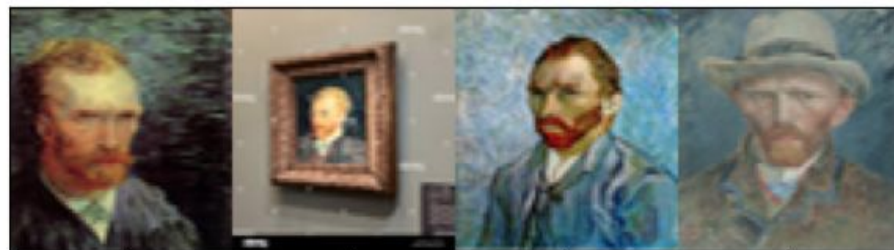


Query

1st

2nd

3rd



Query

1st

2nd

3rd

[Explore the HyperView Google Colab notebook using CLIP and UMAP](#)

- [Data](#)
- [Instructions to add the data to your own Google Drive without downloading](#)

81edf777a63c



Query

1st

2nd

3rd



Query

1st

2nd

3rd

Summary

UMAP reveals structure in high-dimensional embeddings

- Reduces high-dimensional vectors to 2D/3D while preserving local neighborhood structure
- Makes clusters, manifolds, and outliers in embedding spaces visually interpretable

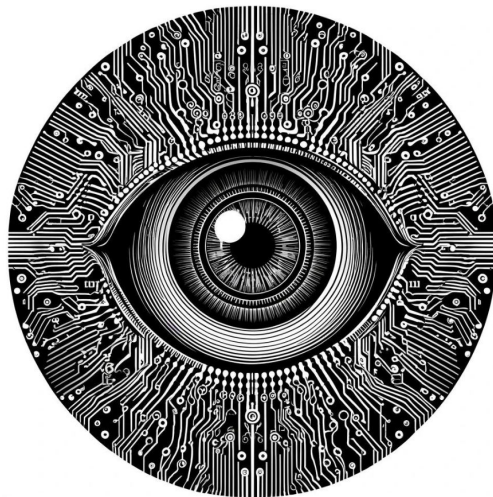
Hyperparameters trade off local detail vs global structure

- `n_neighbors` controls how “local” the notion of similarity is (it is structural)
- `min_dist` controls how tightly points are packed inside clusters (it is aesthetic)

A practical tool for exploring and debugging embeddings

- Helps compare models, spot mislabeled data, and explain embedding-based systems to non-experts

Generative Multimodality with Nemotron 3 Omni



Learning goals

- Understand how combining embeddings create multimodal spaces
- Gain intuitions about the use of multimodal decoders for generation

nemotron-3-nano-omni-30b-a3b-reasoning



What is in this image? ➤



Describe what you see in this video. ➤



00:00

Transcribe the spoken content. ➤



Tools

OFF

ON

Reasoning

Parameters

I can evaluate text, images, audio, or videos based on your prompt or an example from above.



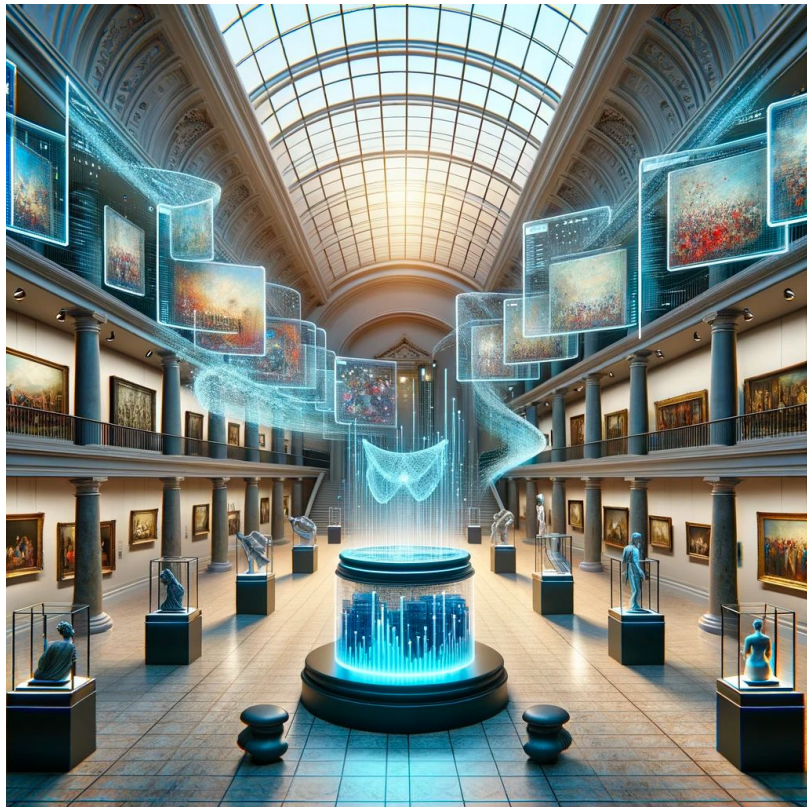
[Live demo at build.nvidia.com](https://build.nvidia.com)

[5 min video walkthrough](#)

Describing Multimodal Data with Nemotron 3 Omni



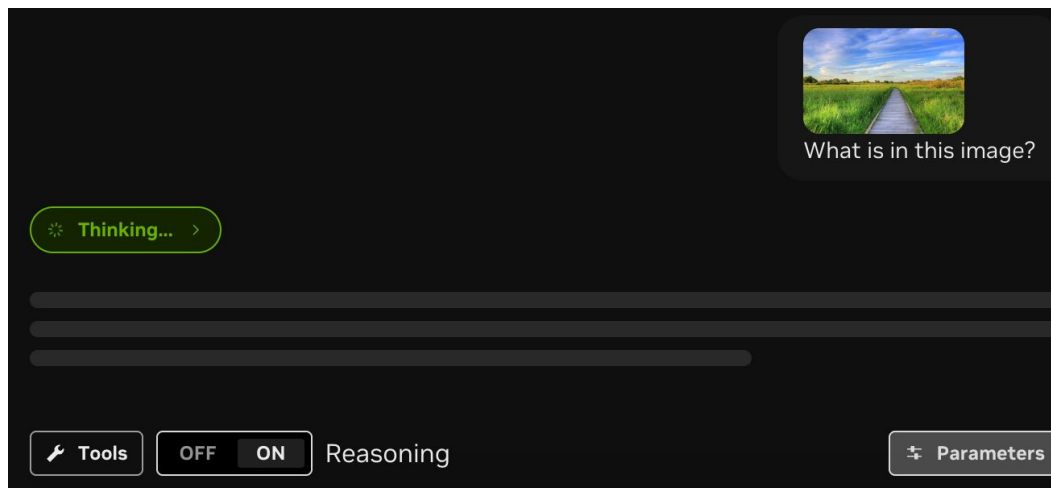
*"A galloping
horse, seen
from the side,
with the jockey
riding it"*



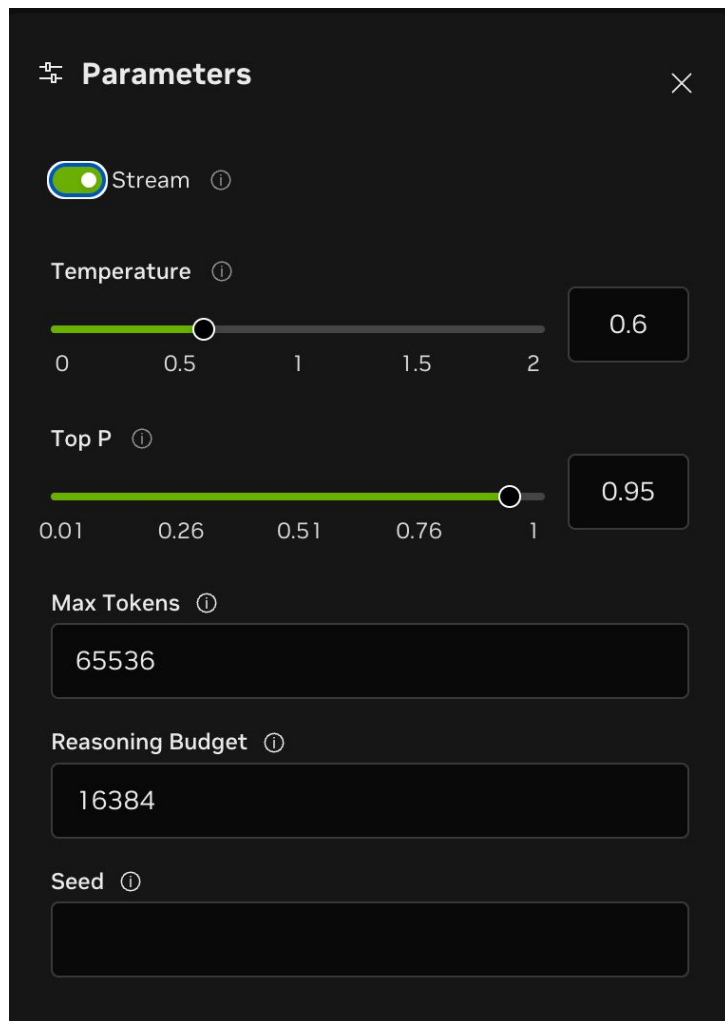
Use cases

- Video search and summarization 🎬
- Multimodal document understanding 📖
- Audio transcription 🗣️
- Dataset curation 🛠️

Understand the architecture to understand its parameters

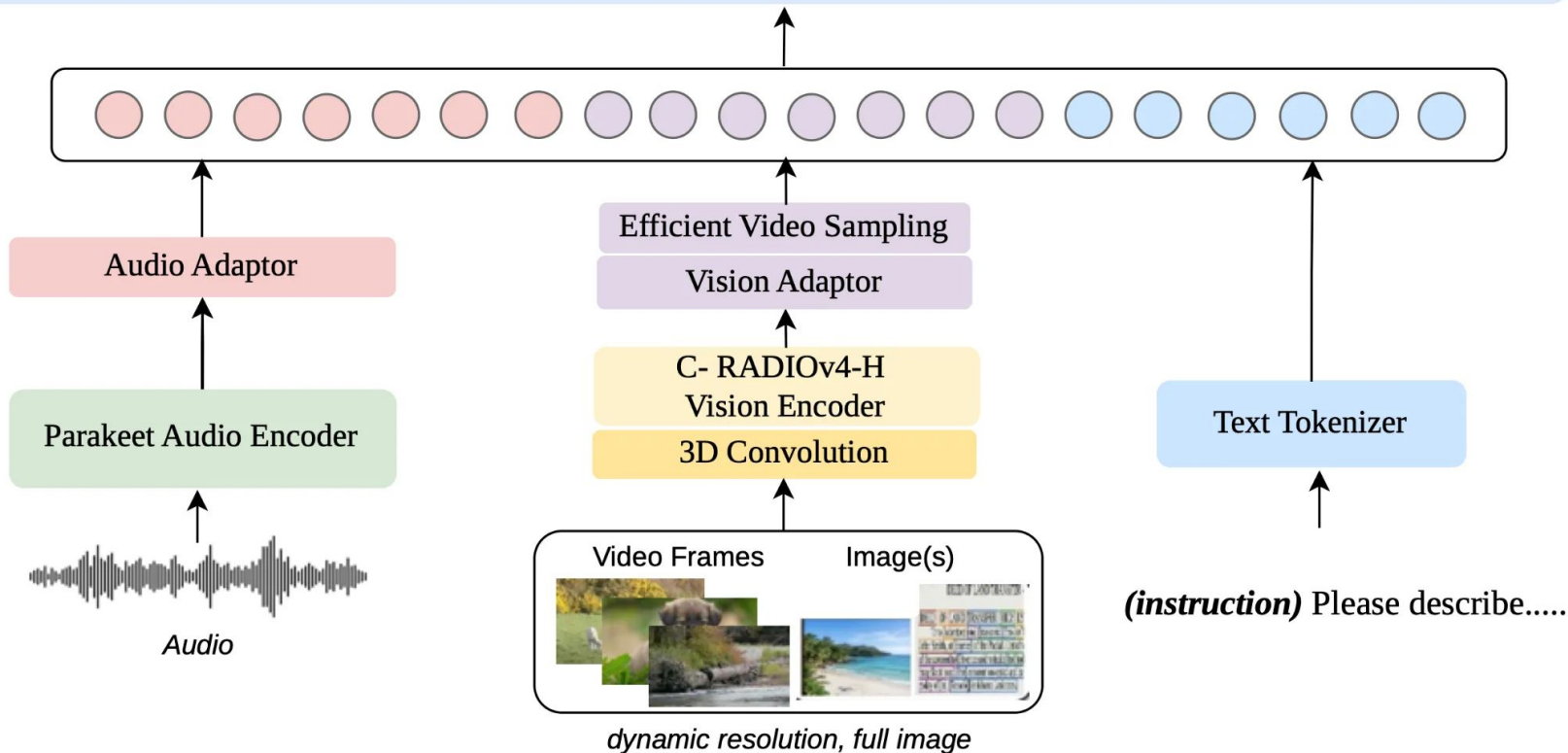


<https://build.nvidia.com/nvidia/nemotron-3-nano-omni-30b-a3b-reasoning>



There is a

Nemotron 3 Nano 30B-A3B LLM



NVIDIA's C-RADIO for Text ↔ Image

Text Prompt: "shoe"



Text Prompt: "helmet"



<https://huggingface.co/nvidia/C-RADIOv4-H>

[C-RADIOv4 \(Tech Report\)](#)

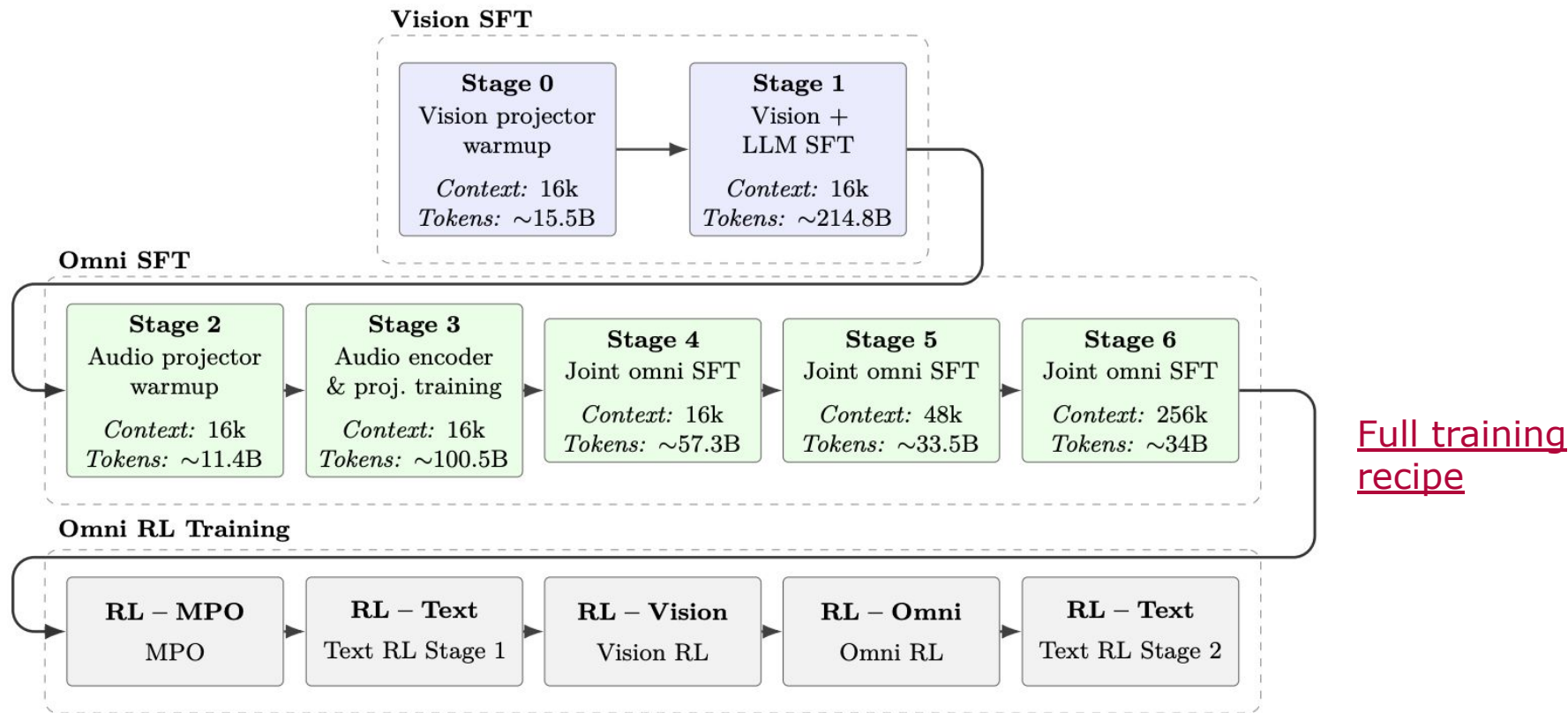


Figure 2 | Staged training recipe for the v3 omni-modal model. The pipeline first performs vision SFT, then joint omni SFT while progressively extending context length, followed by omni-modal RL training.

Long-document visual reasoning

Adobe 2015 10-K • 116-page filing



Handles questions over a 100+ page document

UNITED STATES
SECURITIES AND EXCHANGE COMMISSION
Washington, D.C. 20549

FORM 10-K

(Mark One)

☒ ANNUAL REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE ACT OF 1934
For the fiscal year ended November 27, 2015

☐ TRANSITION REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE ACT OF 1934
For the transition period from _____ to _____

Commission File Number: 0-15175

Adobe
ADOBE SYSTEMS INCORPORATED
(Exact name of registrant as specified in charter)

Delaware
(State or other jurisdiction of incorporation or organization)

77-0818522
(U.S. Employer Identification No.)

345 Park Avenue, San Jose, California 95110-2794
(Address of principal executive office)

(408) 536-6000
(Registrant's telephone number, including area code)

Securities registered pursuant to Section 12(b) of the Act:

Title of Each Class	Name of Each Exchange on Which Registered
Common Stock, \$0.0001 per value per share	The NASDAQ Stock Market LLC (NASDAQ Global Select Market)

Securities registered pursuant to Section 12(g) of the Act: None

DOCUMENTS INCORPORATED BY REFERENCE

Portions of the Proxy Statement for the registrant's 2016 Annual Meeting of Stockholders (the "Proxy Statement"), to be filed within 120 days of the end of the fiscal year ended November 27, 2015, are incorporated by reference in Part III hereof. Except with respect to information specifically incorporated by reference in this Form 10-K, the Proxy Statement is not deemed to be filed as part hereof.



Scans until page 63 to find relevant info



Pages 1–62 scanned
Not containing the needed answer

First relevant evidence

63

ADOBE SYSTEMS INCORPORATED
CONSOLIDATED STATEMENTS OF CASH FLOWS
(In thousands)

	Years Ended		
	November 27, 2015	November 28, 2014	November 29, 2013
Cash flows from operating activities:			
Net cash provided by operating activities	\$ 1,469,502	\$ 1,287,482	\$ 1,351,684
Cash flows from investing activities:			
Net cash used for investing activities	(1,488,358)	(490,707)	(1,177,803)
Cash flows from financing activities:			
Net cash used for financing activities	(200,687)	(597,283)	(539,138)
Effect of foreign currency exchange rates on cash and cash equivalents	(21,297)	(6,448)	(5,243)
Net increase (decrease) in cash and cash equivalents	(240,840)	282,844	(360,496)
Cash and cash equivalents at beginning of year	1,117,400	834,556	1,425,052
Cash and cash equivalents at end of year	\$ 876,560	\$ 1,117,400	\$ 834,556

See accompanying Notes to Consolidated Financial Statements.

63

First relevant evidence

Second relevant evidence

99

ADOBE SYSTEMS INCORPORATED
NOTES TO CONSOLIDATED FINANCIAL STATEMENTS (Continued)

NOTE 16. DEBT

Our debt as of November 27, 2015 and November 28, 2014 consisted of the following (in thousands):

	2015	2014
Notes	\$ 1,387,410	\$ 1,496,778
Fair value of interest rate swap	19,821	14,268
Adjusted carrying value of Notes	1,907,231	1,511,046
Capital lease obligations	—	3,269
Total debt and capital lease obligations	1,907,231	1,514,315
Less: current portion	—	603,229
Debt and capital lease obligations	\$ 1,907,231	\$ 911,086

99

Second relevant evidence

Distant pages
63 → 99
combine evidence
to answer



Page 116
End of filing

Title page (Page 1)

Question

What is Cash Flow to Total Debt Ratio for ADBE in FY2015?

Ground truth

0.77

Prediction

0.77



Reasoning summary

Page 63: FY2015 operating cash flow = \$1,469,502k

Page 99: FY2015 total debt = \$1,907,231k

Compute: $1,469,502 / 1,907,231 = 0.7705$

Final answer: 0.77



How it works



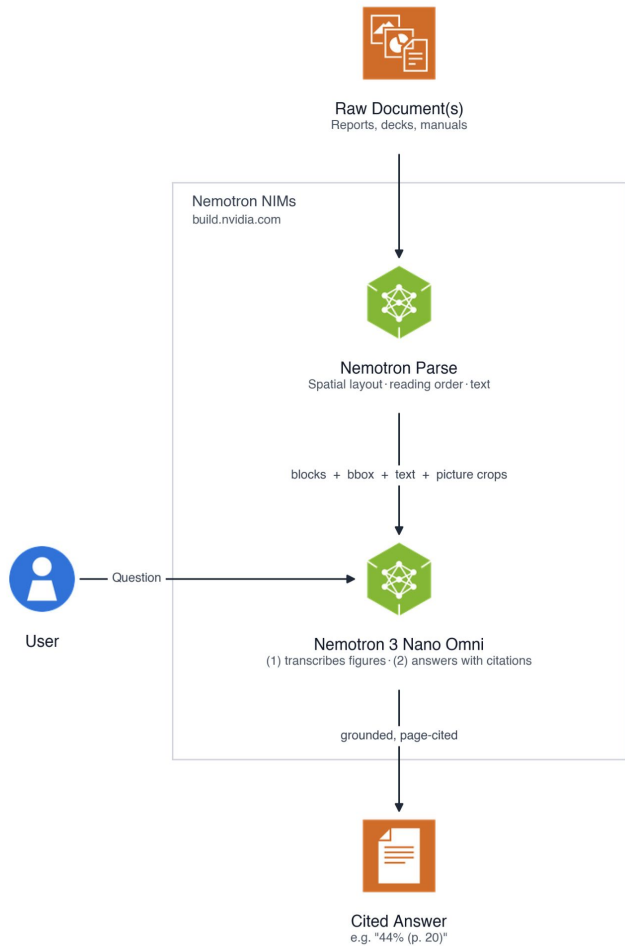
Scans the entire 116-page filing



Finds and extracts relevant evidence



Combines distant evidence to answer



For this sort of task it's best to pair with Nemotron Parse



All-Modality Document AI with Nemotron Parse + Nemotron 3 Nano Omni

You do not need a GPU to run this notebook. Every model call goes to NVIDIA's hosted catalog endpoint at `https://integrate.api.nvidia.com/v1` (the same backend that powers build.nvidia.com). All you need is a free `NVIDIA_API_KEY` — see Setup below for a 30-second walkthrough.

Want to self-host instead? Both models ship as NIMs on NGC -- see the **Deploy** tab on each model card for copy-paste vLLM / SGLang / TensorRT-LLM launch commands ([Parse](#) · [Deploy](#) · [Nano Omni](#) · [Deploy](#)) -- and as open weights (BF16 / FP8 / NVFP4) on Hugging Face. Tested hardware: H100, B200, DGX Spark, RTX Pro 6000, Jetson Thor.

This notebook builds a streamlined, **all-modality** document analysis pipeline by pairing two NVIDIA models. The star of the pipeline is **Nemotron 3 Nano Omni** (`nvidia/nemotron-3-nano-omni-30b-a3b-reasoning`) — a single multimodal reasoning model that both **reads every picture on the page** and **answers questions about the whole document**, keeping what the picture said and what the question needs inside one brain. Alongside it, **Nemotron Parse** (`nvidia/nemotron-parse`) provides the **spatial anchoring** that lets Nano Omni focus on one region at a time and cite the exact page and phrase its answer came from.

To make the case concretely, we run the pipeline against four pages picked from **three different public PDFs** — a Pew Research report, a social-media analytics deck, and a graduate-studies brochure — each chosen to stress a **different content modality** that a single VLM cannot handle alone:

https://github.com/NVIDIA-NeMo/Nemotron/blob/main/usage-cookbook/Nemotron-3-Nano-Omni/doc-intelligence-with-parse/doc_intelligence_cookbook.ipynb

Summary

Nemotron 3 Omni unifies text, vision, and audio in a single model

- Embeddings enable flexible multimodal workflows
- Two modes: descriptive and reasoning

Powers end-to-end pipelines for document intelligence, multimodal search, and assistants

- Multimodal prompts let the model attend to specific regions, layouts, or visual details
- Integrates with tools like Nemotron Parse and RAG to build production-ready multimodal apps

Resources

- [Nemotron 3 Nano Omni - Technical Report](#)
- [Nemotron Developer repository at GitHub](#)
- [Nemotron 3 Nano Omni 30B @ HuggingFace](#)
- [Nemotron Parse 1.2 @ HuggingFace](#)
- [Practical Computer Vision Bootcamp - GitHub Repository](#)